QB-TimeWarp — Project Context & Memory

Project Overview

QB-TimeWarp is a QuickBooks data migration tool that exports data from **QuickBooks 2023 Premier Accountant Edition** and imports it into **QuickBooks 2021**. The project exists because the client's accountant requires all historical data to be accessible in QB 2021 for ongoing work, and QuickBooks does not natively support backward migration (only forward upgrades).

- **Source**: QuickBooks 2023 Premier Accountant (`Air-Masters-QB-2023.qbw`, ~360 MB)
- **Target**: QuickBooks 2021 (`Josh Safty 2021.qbw`, ~13 MB)
- **Technology**: C# .NET 6.0 (x86, Windows-only), QBXML SDK
- **Repository**: https://github.com/LetsLearnAboutIt/QuickBooks-TimeWarp
- **Deployment Target**: `C:\QB-TimeWarp` on a Windows VM

Stakeholder Requirements & Reasoning

#	Requirement	WHY (Business Justification)
1	**Export ALL entities including inactive**	Ensure no data loss during migration. Inactive customers, vendors, items, etc. must be preserved so historical transactions referencing them remain valid.
2	**Convert inactive entities to active in QB 2021**	Accountant requirement for a clean migration. QB 2021 import may reject or skip inactive entities; reactivating them ensures all records import successfully. The accountant can re-deactivate as needed post-migration.
3	**Preserve class tracking**	Maintain cost center reporting. Classes in QB are used to track departments/locations/projects. Losing class assignments would break the accountant’s financial reports and P&L by class.
4	**Match accounting model (Cash/Accrual)**	Financial reporting consistency. If QB 2023 uses Accrual basis and QB 2021 uses Cash, all financial reports would be incorrect. The accounting method must match.
5	**Preserve date/field formats**	Data integrity. Dates must remain in QBXML-compatible format (yyyy-MM-dd). Currency must maintain decimal precision. Phone numbers, postal codes (leading zeros), and memo fields must not be corrupted during transformation.
6	**Journal matching validation (debits = credits)**	Accountant’s critical requirement. Every journal entry must balance — total debits must equal total credits. Un-

#	Requirement	WHY (Business Justification)
		balanced journals indicate data corruption and would be rejected by QB 2021 or produce incorrect financial statements.

Design Decisions & Rationale

1. QBXML SDK (not IIF or direct file manipulation)

- **Decision**: Use QBXML request/response SDK via COM Interop
- **Why**: QBXML is the officially supported QuickBooks SDK for programmatic access. IIF import is deprecated and unreliable for complex data. Direct `.qbw` file manipulation is undocumented and risky.

2. x86 Platform Target

- **Decision**: Target `win-x86` / `PlatformTarget=x86`
- **Why**: The QuickBooks QBXML SDK COM libraries are 32-bit only. A 64-bit process cannot load 32-bit COM objects.

3. Export → Transform → Import → Validate Pipeline

- **Decision**: Four-stage pipeline with intermediate JSON files
- **Why**: Allows inspection of data at each stage. If import fails, you don't need to re-export. Each stage can be re-run independently. JSON is human-readable for debugging.

4. JSON as Intermediate Format

- **Decision**: Export to JSON, transform JSON, import from JSON
- **Why**: JSON is easier to inspect, diff, and debug than XML. Newtonsoft.Json provides excellent JObject manipulation. The QBXML conversion happens at the boundary (export/import), not in the middle.

5. Inactive → Active Transformation (not filtering)

- **Decision**: Transform inactive entities to active instead of filtering them out
- **Why**: Filtering would lose data. The accountant needs all records present. Reactivation ensures QB 2021 accepts them. This is a reversible operation.

6. Configuration-Driven Behavior

- **Decision**: All major behaviors controlled by `appsettings.json` and `FieldMappings.json`
- **Why**: Allows the accountant or operator to toggle features without code changes. Different migrations may need different settings.

7. Import Order (Reference entities first)

- **Decision**: Import in dependency order — Accounts → Terms → Classes → Customers → Vendors → Items → Transactions

- **Why**: Transactions reference customers, vendors, items, and accounts. Those must exist first or the import will fail with “not found” errors.

8. Journal Validation as a Separate Step

- **Decision**: Validate journal entries post-import as a distinct validation step
- **Why**: Journal balance is a critical accounting invariant. Checking it separately ensures it gets explicit pass/fail status in reports.

Transformation Rules (Business Justification)

Rule	Config Key	What It Does	Business Justification
Reactivate Inactive Entities	`ReactivateInactiveEntities`	Sets `IsActive=true` on all entities during transform	Ensures QB 2021 accepts all records; accountant can deactivate later
Preserve Class Tracking	`PreserveClassTracking`	Maintains class assignments; auto-creates missing classes in QB 2021	Preserves cost center / department reporting
Match Accounting Model	`MatchAccountingModel`	Queries source accounting method and applies it to target	Ensures Cash/Accrual basis matches for accurate financial reports
Preserve Date Formats	`format-Rules.preserveDateFormat`	Converts dates to QBXML standard (yyyy-MM-dd), strips timezones	QB 2021 doesn’t support timezone-aware dates
Preserve Currency Precision	`format-Rules.preserveCurrencyFormat`	Maintains 2 decimal places for amounts	Prevents rounding errors in financial data
Preserve Phone Formats	`format-Rules.preservePhoneFormat`	Keeps formatting (dashes, parens) within 21-char QB limit	Data integrity for contact information
Preserve Postal Codes	`format-Rules.preservePostalCodeFormat`	Preserves leading zeros within 13-char QB limit	Critical for US ZIP codes starting with 0 (e.g., 01234)
Preserve Memo Formatting	`format-Rules.preserveMemoFormatting`	Normalizes line breaks, preserves UTF-8	Prevents data corruption in notes/descriptions

Collaborative Development Approach

> **This project is developed collaboratively between the AI agent and a human user who is actively involved.**

How We Work Together

Principle	Details
User is actively watching	The user can see the VM screen via RDP screen sharing and monitors QuickBooks windows during execution
Check in regularly	During long operations (export, import), pause and share progress with the user before proceeding
User handles popups	QuickBooks generates popups (permissions, passwords, warnings) that may need human intervention — the user can see and respond to these
Don’t barrel through	At each pipeline stage (Export → Transform → Import → Validate), stop and coordinate with the user
User provides context	The user knows the QuickBooks data and can identify if results look wrong — leverage their expertise
Shared debugging	When errors occur, share the error with the user — they may recognize the issue faster from the QB UI

When to STOP and Check with User

1. **Before starting any pipeline stage** — confirm we’re ready
2. **After export completes** — verify entity counts look reasonable
3. **Before import** — this writes to QB 2021, confirm transformation looks correct
4. **When any unexpected popup appears** — user can see and handle it
5. **When errors occur** — share the error, user may have context
6. **After validation** — review results together

Key Operational Notes

- **Keep QuickBooks windows VISIBLE** — popups can block SDK operations silently if windows are minimized
- **Passwords are in** `PASSWORDS.txt` — enter when QuickBooks prompts for company file passwords
- **See** `POPUP_HANDLING.md` — comprehensive guide to common QuickBooks popups and how to handle them

Key Files & Their Purposes

File	Purpose
`Program.cs`	Main orchestrator — runs the 4-stage pipeline
`Services/QBConnectionManager.cs`	Manages QBXML SDK COM connections to QuickBooks
`Services/DataExporter.cs`	Exports data from QB 2023 (with ActiveStatus=All)
`Services/DataTransformer.cs`	Transforms exported data for QB 2021 compatibility
`Services/DataImporter.cs`	Imports transformed data into QB 2021
`Services/DataValidator.cs`	Validates migrated data (field comparison, journal balance, formats)
`Services/SchemaExtractor.cs`	Extracts QB 2021 field schemas for mapping
`Helpers/TransformFunctions.cs`	Utility functions for format preservation
`Helpers/ProgressReporter.cs`	Console progress bars and banners
`Models/ConfigurationModels.cs`	Configuration POCO classes
`Models/MigrationModels.cs`	Data models for export/import/transform
`Models/ValidationModels.cs`	Validation report models
`Models/FieldMappingModels.cs`	Field mapping and format rule models
`Models/QBEntityType.cs`	Entity type definitions
`appsettings.json`	Main application configuration
`Configuration/FieldMappings.json`	Field mappings and format rules

Entity Types Handled

Reference/Setup Entities: Accounts, PaymentMethods, Terms, Classes, SalesTaxCodes, ShipMethods, CustomerTypes, VendorTypes, JobTypes, PriceLevels

Party Entities: Customers, Vendors, Employees

Item Entities: Items (Service, Inventory, Non-Inventory, etc.)

Transaction Entities: Invoices, Bills, Payments, SalesReceipts, PurchaseOrders, JournalEntries, CreditMemos, Estimates, Deposits, Checks, VendorCredits, InventoryAdjustments, Transfers

System Entities: Preferences, CompanyInfo

Future Enhancements

GUI Application with Drag-and-Drop Interface

- **Phase**: Future development (post-CLI stabilization)
- **Concept**: Build a desktop GUI application where the customer can simply drag-and-drop their `.QBW` file directly onto the app window
- **Benefits**:
- Eliminates need for customers to edit `appsettings.json` or use command-line arguments
- Intuitive workflow: drag source `.QBW` → drag target `.QBW` → click “Migrate”
- Visual progress indicators, real-time log viewer, and validation summary dashboard
- **Technical Notes**:
- Consider WPF or WinForms for Windows-native drag-and-drop support
- Reuse existing service layer (`DataExporter` , `DataTransformer` , `DataImporter` , `DataValidator`)
- GUI wraps the same pipeline — no migration logic changes needed
- Working copy system operates transparently behind the GUI
- **Priority**: After CLI pipeline is fully tested and deployed